

FUNÇÕES, MÓDULOS E ORGANIZAÇÃO DE CÓDIGO

ESCREVENDO PROGRAMAS MAIS ORGANIZADOS E REUTILIZÁVEIS



Caroline Perez Gonçalves
2026

FUNÇÕES, MÓDULOS E ORGANIZAÇÃO DE CÓDIGO

Caroline Perez Gonçalves

SUMÁRIO

APRESENTAÇÃO.....	1
INTRODUÇÃO	2
CRIANDO FUNÇÕES	3
REUTILIZANDO UMA FUNÇÃO	4
PARÂMETROS.....	5
UTILIZANDO VÁRIOS PARÂMETROS.....	6
POR QUE UTILIZAR PARÂMETROS?	6
RETORNO (RETURN)	7
ESCOPO DE VARIÁVEIS.....	9
MÓDULOS	11
POR QUE UTILIZAR MÓDULOS?	12
IMPORTAÇÃO DE BIBLIOTECAS.....	13
IMPORTANDO APENAS UMA FUNÇÃO.....	14
BIBLIOTECAS	14
BIBLIOTECAS PADRÃO.....	15
TRATAMENTO DE ERROS (TRY E EXCEPT)	19
PROJETO PRÁTICO	23
AGRADECIMENTO.....	28
CONTATO.....	29

APRESENTAÇÃO

Aos que não me conhecem, meu nome é Caroline. Sou formada em Análise e Desenvolvimento de Sistemas, pós-graduada em Inteligência Artificial e Machine Learning e possuo MBA em Gerenciamento de Projetos.

Além da minha experiência profissional com tecnologia, projetos e equipes multidisciplinares, também atuo como instrutora. Sempre acreditei que o conhecimento deve ser compartilhado de forma simples, leve e acessível, e foi com esse propósito que esta apostila foi desenvolvida.

Se você chegou até aqui, parabéns! Isso significa que já construiu uma boa base em Python. Agora é o momento de aprender a escrever códigos mais organizados, reutilizáveis e próximos da realidade do desenvolvimento profissional.

Ao longo desta apostila, você conhecerá conceitos fundamentais, como funções, módulos, bibliotecas e tratamento de erros, recursos que tornarão seus programas mais limpos, eficientes e fáceis de manter.

Vamos aos estudos! Mas, antes, sente-se, relaxe e sinta-se em casa. Quero que este material continue sendo como uma boa conversa entre amigos, tornando o aprendizado mais leve, agradável e, principalmente, significativo.

Boa sorte e muito sucesso!

INTRODUÇÃO

Até aqui, você aprendeu os principais fundamentos da linguagem Python. Criou seus primeiros programas, trabalhou com estruturas condicionais, laços de repetição e conheceu diferentes formas de organizar informações utilizando estruturas de dados.

Mas existe uma pergunta importante.

Como desenvolver programas maiores sem que o código fique desorganizado e difícil de manter?

A resposta para essa pergunta está na forma como escrevemos nossos programas. À medida que um projeto cresce, copiar e colar trechos de código deixa de ser uma boa prática. O desenvolvimento passa a exigir organização, reutilização de código e uma estrutura que facilite futuras alterações.

Como assim, Carol?

Imagine um sistema com centenas ou até milhares de linhas de código. Fazer qualquer alteração pode se tornar uma tarefa complicada se tudo estiver escrito em um único arquivo. Por isso, desenvolvedores utilizam recursos como funções, módulos e bibliotecas para dividir o programa em partes menores, tornando o código mais organizado, reutilizável e fácil de entender.

E tem mais: escrever um bom programa não significa apenas fazer com que ele funcione. Significa desenvolver soluções claras, organizadas e preparadas para crescer conforme novas necessidades surgem.

Ao longo desta apostila, você aprenderá técnicas que fazem parte da rotina de qualquer desenvolvedor Python e que serão fundamentais para criar aplicações cada vez mais profissionais.

Para que esta introdução não fique muito extensa, vamos explorar cada assunto no momento certo. E, como sempre, vamos começar pelo começo.

Vamos lá.

CRIANDO FUNÇÕES

Acredito que podemos concordar que os nossos programas cresceram bastante.

Aprendemos a receber informações do usuário, realizar cálculos, tomar decisões, utilizar laços de repetição e trabalhar com diferentes estruturas de dados.

Mas pense agora que você desenvolveu um sistema para uma escola e, em vários momentos do programa é necessário exibir o mesmo cabeçalho.

```
=====
```

```
SISTEMA ESCOLAR
```

```
=====
```

Uma solução seria copiar esse código para todos os lugares onde ele fosse necessário.

Funciona? Sim.

Mas imagine que, alguns dias depois, a escola decida alterar o nome do sistema.

Você teria que procurar todos os trechos do programa para fazer a mesma alteração.

Além de trabalhoso e chato, isso aumenta a chance de cometer erros. É justamente para resolver esse tipo de situação que existem as **funções**.

Uma função é um bloco de código criado para executar uma tarefa específica.

Depois de criada, ela pode ser utilizada quantas vezes forem necessárias, evitando repetição de código e tornando o programa muito mais organizado.

Criando a primeira função

Em Python, utilizamos a palavra-chave **def** para criar uma função. Veja um exemplo:

```
def saudacao():
```

```
    print("Bem-vindo ao sistema!")
```

Observe que, nesse momento, a função foi apenas criada. Ela ainda não foi executada. Para utilizá-la, basta informar seu nome seguido de parênteses.

```
def saudacao():
```

```
    print("Bem-vindo ao sistema!")
```

```
saudacao()
```

Resultado: Bem-vindo ao sistema!

Sempre que chamarmos a função, o código existente dentro dela será executado.

REUTILIZANDO UMA FUNÇÃO

Uma das maiores vantagens das funções é a reutilização. Veja o exemplo:

```
def saudacao():
```

```
    print("Bem-vindo ao sistema!")
```

```
saudacao()
```

```
saudacao()
```

```
saudacao()
```

Resultado:

Bem-vindo ao sistema!

Bem-vindo ao sistema!

Bem-vindo ao sistema!

Em vez de escrever a mesma instrução várias vezes, basta chamar a função sempre que necessário. Simples, né?

Um exemplo do dia a dia

Em um sistema bancário., sempre que o cliente realizar uma operação, o sistema deverá exibir uma mensagem de confirmação.

Sem funções, esse mesmo código precisaria ser repetido diversas vezes.

Com uma função, basta criar essa mensagem uma única vez e reutilizá-la em todo o sistema.

Esse é apenas um dos inúmeros exemplos de como as funções ajudam a organizar programas maiores.

Utilizar funções oferecem diversas vantagens. Com elas, você evita repetição de código, elas tornam os programas mais organizados, facilitam a manutenção, permitem reutilizar uma mesma tarefa várias e várias vezes e deixam a leitura do código muitooo mais simples.

Por esse motivo, praticamente todos os programas profissionais utilizam funções.

PARÂMETROS

Já que vimos a criação de funções, nada mais justo do que seguirmos com os parâmetros.

A função vista anteriormente sempre exibía exatamente a mesma mensagem.

```
def saudacao():
```

```
    print("Bem-vindo ao sistema!")
```

Mas será que faz sentido criar uma função diferente para cumprimentar cada pessoa?

Digo criar algo como:

```
def ola_ana():
```

```
def ola_carlos():
```

```
def ola_fernanda():
```

Faz sentido isso?

Claro que não.

Então, precisamos de uma forma para que a mesma função possa trabalhar com diferentes informações.

É justamente para isso que existem os **parâmetros**.

Os parâmetros permitem enviar dados para uma função no momento em que ela é executada. Isso torna as funções muito mais reutilizáveis e é bem simples. Vamos ver abaixo.

Criando uma função com parâmetro

Veja o exemplo abaixo:

```
def saudacao(nome):
```

```
    print("Olá, ", nome)
```

Observe que agora a função possui um parâmetro chamado **nome**. Para executá-la, precisamos informar um valor.

```
saudacao("Caroline")
```

Resultado: Olá, Caroline

Também podemos chamá-la novamente com outro valor.


```
saudacao("João")
```

```
saudacao("Fernanda")
```

Resultados:

Olá, João

Olá, Fernanda

Perceba que a função continua sendo a mesma. O que muda é apenas a informação recebida.

UTILIZANDO VÁRIOS PARÂMETROS

Uma função pode receber mais de uma informação. Veja outro exemplo:

```
def apresentar(nome, idade):  
    print(nome, "possui", idade, "anos.")
```

Executando a função.

```
apresentar("Lucas", 25)
```

Resultado: Lucas possui 25 anos.

Cada parâmetro representa uma informação diferente que será utilizada pela função.

Um exemplo do dia a dia

Imagine um sistema de uma loja. Sempre que uma venda é concluída, o programa precisa emitir uma mensagem como:

Pedido nº 1523 aprovado.

Criar uma função diferente para cada pedido não faria sentido. Em vez disso, criamos uma única função que recebe o número do pedido como parâmetro.

Assim, ela poderá ser utilizada para qualquer venda realizada.

POR QUE UTILIZAR PARÂMETROS?

Os parâmetros tornam as funções muito mais flexíveis. Com eles, podemos utilizar a mesma função em diferentes situações, alterando apenas as informações enviadas.

Isso reduz a quantidade de código, facilita a manutenção e torna os programas muito mais organizados. E a organização é um dos pontos mais importantes dentro de um código, vai por mim.

RETORNO (RETURN)

Criamos funções e vimos que elas podem receber diversos parâmetros. Mas agora surgiu uma dúvida.

Será que uma função consegue devolver alguma informação?

A resposta é sim!

E é justamente para isso que existe o **return**.

Mas calma... antes de olhar qualquer código, vamos imaginar uma situação (me perdoem por escrever isso de novo).

Suponha que você criou uma função para calcular o valor total de uma compra.

Ela recebe dois preços e faz a soma.

E depois? Como esse resultado volta para o programa?

É aí que entra o return.

Ele serve para **devolver um valor** para quem chamou a função.

Pense na função como uma pessoa. Você faz uma pergunta ela realiza um trabalho e depois responde.

Essa resposta é exatamente o que o return entrega.

Primeiro exemplo

Vamos criar uma função que soma dois números.

```
def somar(numero1, numero2):  
    resultado = numero1 + numero2  
    return resultado
```

Até aqui apenas criamos a função. Agora vamos utilizá-la:

```
total = somar(10, 5)  
print(total)
```

Resultado: 15

Percebeu o que aconteceu? A função fez o cálculo e devolveu o resultado para a variável total.

Depois utilizamos essa variável normalmente.

Mas... por que não usar apenas print()?

Essa é uma dúvida muito comum. E, sinceramente, muita gente confunde essas duas coisas no começo (acho que é normal até ter um delay em um dia qualquer e pensar sobre isso aleatoriamente). Olha só.

```
def somar(numero1, numero2):  
    print(numero1 + numero2)
```

Esse código funciona. Ele exibirá: 15

Mas existe um detalhe importante. O valor foi apenas mostrado na tela. Ele **não voltou para o programa**. Agora veja usando return.:

```
def somar(numero1, numero2):  
    return numero1 + numero2
```

Percebe a diferença?

Agora podemos guardar esse resultado em uma variável. Podemos fazer novos cálculos. Podemos enviar esse valor para outra função. Ou seja...

O programa continua trabalhando com aquela informação.

Uma dica que costuma ajudar é esta:

print() **mostra** uma informação.

return **devolve** uma informação.

Pode parecer uma diferença pequena, mas ela muda completamente a forma como escrevemos nossos programas. Lembre-se disso, ok?

Outro exemplo

Imagine um sistema que calcula a média de um aluno.

```
def calcular_media(nota1, nota2):  
    media = (nota1 + nota2) / 2  
    return media
```

Agora podemos utilizar essa média em qualquer lugar.

```
resultado = calcular_media(8, 6)

print(resultado)
```

Resultado: 7.0

Um exemplo do dia a dia

Imagine uma calculadora. Você digita: $2 + 3$ e ela mostra: 5

Mas, internamente, ela primeiro faz o cálculo e depois entrega esse resultado.

É exatamente isso que uma função faz quando utiliza `return`.

Ela realiza uma tarefa e devolve o resultado para o restante do programa.

ESCOPO DE VARIÁVEIS

Vou começar este capítulo dizendo uma coisa (e não é “imagine”).

Esse assunto costuma assustar quem está começando. Mas relaxa...

Depois que você entende a ideia, percebe que ele é muito mais simples do que parece.

Pensa assim:

Você trabalha em uma empresa. Cada funcionário possui sua própria mesa. Os objetos que estão sobre a sua mesa pertencem ao seu espaço de trabalho. Quem está em outra sala normalmente não tem acesso àquilo.

Com as variáveis acontece algo parecido. Nem toda variável pode ser utilizada em qualquer lugar do programa. Tudo depende de onde ela foi criada.

É isso que chamamos de **escopo**.

Variável local

Veja este exemplo abaixo:

```
def saudacao():

    mensagem = "Olá!"

    print(mensagem)
```

Até aqui tudo funciona normalmente. Mas agora observe:

```
def saudacao():  
    mensagem = "Olá!"  
print(mensagem)
```

O que acontecerá?

.
.
.

O Python apresentará um erro.

Como assim, Carol?

A variável mensagem foi criada **dentro** da função. Por isso, ela existe apenas naquele espaço. Quando a função termina, aquela variável deixa de estar disponível.

Esse tipo de variável recebe o nome de **variável local**.

Ou seja... Ela pertence apenas à função onde foi criada.

Variável global

Agora veja outro exemplo.

```
mensagem = "Bem-vindo!"  
  
def saudacao():  
    print(mensagem)  
  
saudacao()
```

Resultado: Bem-vindo!

Percebeu a diferença?

Agora a variável foi criada **fora** da função. Isso significa que ela pode ser utilizada em diferentes partes do programa.

Chamamos esse tipo de variável de **variável global**.

Então devo criar tudo como variável global?

Não. Na verdade, o ideal é fazer exatamente o contrário.

Sempre que possível, utilize variáveis locais.

Eita! Como assim?

Imagine uma empresa onde qualquer funcionário pudesse alterar qualquer documento (provavelmente seria uma grande bagunça).

Na programação acontece algo parecido. Quanto menor o alcance de uma variável, mais organizado tende a ficar o código.

Por isso, normalmente criamos variáveis apenas onde elas realmente são necessárias.

Um exemplo do dia a dia

Imagine uma receita de bolo. Os ingredientes utilizados naquela receita pertencem apenas àquele preparo.

Eles não precisam estar disponíveis para todas as receitas do livro. As funções funcionam da mesma maneira.

Cada uma possui suas próprias informações. E isso evita conflitos e torna o programa muito mais organizado.

MÓDULOS

Até aqui, todos os nossos programas foram escritos em um único arquivo.

No começo isso não representa nenhum problema. Mas deixa eu te fazer uma pergunta.

Imagine um sistema com cinco, dez ou até cinquenta mil linhas de código.

Você acha que encontrar uma informação nesse arquivo seria uma tarefa fácil?

Eu não conseguiria fácil. Aliás... seria um verdadeiro pesadelo.

É justamente por isso que existem os **módulos**.

Um módulo nada mais é do que um arquivo Python que contém código que pode ser reutilizado em outros programas.

Em outras palavras, em vez de colocar tudo em um único arquivo, podemos dividir nosso projeto em várias partes menores e mais organizadas.

Pense em um guarda-roupa. Você poderia jogar todas as roupas em uma única gaveta?

Poderia (inclusive, eu sou tão pecadora com organização do meu guarda-roupa. Nem eu acredito que consigo ser tão desleixada).

Mas encontrar uma camiseta depois seria bem complicado (e é mesmo. Posso dizer isso com propriedade).

Normalmente temos que organizar tudo por categorias.

- Uma gaveta para camisetas.
- Outra para calças.
- Outra para roupas de frio.

Com os programas acontece exatamente a mesma coisa. Cada arquivo fica responsável por uma parte do projeto.

Isso deixa o desenvolvimento muito mais organizado.

Criando um módulo

Vamos imaginar que criamos um arquivo chamado **mensagens.py**. Dentro dele existe uma função.

```
def boas_vindas():  
    print("Seja bem-vindo ao sistema!")
```

Esse arquivo agora é um módulo e ele poderá ser utilizado por outros programas sempre que necessário. Simples assim.

POR QUE UTILIZAR MÓDULOS?

Pode parecer exagero dividir um programa pequeno em vários arquivos. E, sinceramente, no começo realmente parece.

Mas já imaginou um sistema desenvolvido por uma equipe com dez programadores? Se todo mundo escrever no mesmo arquivo, rapidamente o projeto ficará desorganizado.

Os módulos ajudam justamente a evitar isso. Eles permitem dividir um sistema em pequenas partes, pedacinhos mesmo, tornando o código mais limpo, organizado e fácil de manter.

Um exemplo do dia a dia

Dentro de uma oficina mecânica, existe um profissional especializado em motores,

outro em suspensão e outro em elétrica. Cada um cuida da sua área.

Agora imagine se todos trabalhassem ao mesmo tempo no mesmo espaço, mexendo nas mesmas peças. Seria uma enorme confusão.

Com os módulos acontece algo parecido.

Cada arquivo possui uma responsabilidade e isso torna o projeto muito mais organizado.

Complete a frase: “A organização é ...”.

Importante!

IMPORTAÇÃO DE BIBLIOTECAS

Se podemos dividir nossos programas em pequenos pedaços, onde chamamos de “módulos”, como podemos utilizar uma função que está em outro arquivo?

A resposta também é simples. Nós a **importamos**.

Aliás... Se eu pudesse decidir isso, diria que esse é um dos maiores poderes do Python.

Você não precisa escrever tudo do zero.

Existe uma enorme quantidade de códigos prontos esperando para serem utilizados. E isso economiza muito tempo. Tempo é vida!

Importando um módulo

Vamos imaginar novamente o arquivo **mensagens.py**.

```
def boas_vindas():  
    print("Seja bem-vindo ao sistema!")
```

Agora, em outro arquivo, podemos utilizá-lo assim.

```
import mensagens  
  
mensagens.boas_vindas()
```

Resultado: Seja bem-vindo ao sistema!

Percebeu o que aconteceu?

Mesmo estando em outro arquivo, conseguimos utilizar a função normalmente.

Isso acontece porque fizemos a importação do módulo.

IMPORTANDO APENAS UMA FUNÇÃO

Também podemos importar apenas o que realmente precisamos. Dê uma olhada abaixo:

```
from mensagens import boas_vindas
```

```
boas_vindas()
```

O resultado será exatamente o mesmo. A diferença é que agora importamos apenas uma função, e não o módulo inteiro.

BIBLIOTECAS

Agora vem uma informação muito interessante.

O Python possui centenas de bibliotecas prontas.

Na verdade... M-I-L-H-A-R-E-S.

Essas bibliotecas resolvem problemas que outros desenvolvedores já solucionaram. Então, em vez de reinventar a roda, basta utilizá-las.

Quer trabalhar com datas? Existe uma biblioteca para isso.

Quer gerar números aleatórios? Também existe.

Quer criar interfaces gráficas? Manipular imagens? Desenvolver Inteligência Artificial? Consumir APIs?

Pode acreditar que, muito provavelmente já existe uma biblioteca pronta para ajudar.

E nós veremos várias delas ao longo da trilha.

Um exemplo do dia a dia

Imagine uma construção... Você poderia fabricar cada tijolo antes de levantar uma parede.

Mas isso faria sentido? Claro que não.

É muito mais inteligente utilizar materiais que já estão prontos. Já pensou fabricar um por um?

As bibliotecas seguem exatamente essa ideia.

Elas oferecem soluções prontas para problemas muito comuns.

Assim você pode concentrar seus esforços naquilo que realmente importa: desenvolver sua aplicação.

BIBLIOTECAS PADRÃO

Até aqui, aprendemos a criar nossas próprias funções e também vimos que podemos organizar nossos projetos utilizando módulos.

Mas deixa eu te fazer uma pergunta.

Imagine que você precise desenvolver um programa que mostre a data e a hora atual. Você escreveria toda a lógica para descobrir o dia, o mês, o ano, as horas, os minutos e os segundos? Provavelmente não, né?

Agora imagine precisar gerar um número aleatório.

Ou fazer um cálculo matemático mais complexo.

Ou copiar um arquivo.

Será que precisamos desenvolver tudo isso do zero?

A resposta é **não**. Não, não e não!

E essa é uma das grandes vantagens do Python. Ele já vem acompanhado de uma enorme coleção de ferramentas prontas, conhecidas como **bibliotecas padrão**.

Pense nelas como uma caixa de ferramentas (é o único exemplo que acho bacana e bem ligado).

Você não constrói uma chave de fenda toda vez que precisa apertar um parafuso. Você simplesmente pega a ferramenta certa e utiliza.

As bibliotecas padrão funcionam exatamente assim.

Elas oferecem soluções prontas para problemas muito comuns, permitindo que você economize tempo e concentre seus esforços no que realmente importa: desenvolver sua aplicação ou chorar na frente do computador.

O que são bibliotecas padrão?

As bibliotecas padrão são conjuntos de módulos que acompanham a instalação do Python. Isso significa que você pode utilizá-las imediatamente, sem precisar instalar mais nada. Olha só que legal!

Você instala o Python hoje e já recebe centenas de recursos prontos para trabalhar com datas e horários, operações matemáticas, arquivos, números aleatórios, manipulação de textos, sistemas operacionais, entre muitas outras funcionalidades.

Pode parecer pouco agora. Mas, conforme seus projetos crescerem e sua rotina ficar maluca, você perceberá o quanto essas bibliotecas facilitam o desenvolvimento. Bora conhecer algumas agora!

A biblioteca math

Vamos começar por uma das mais conhecidas. Imagine que você precise calcular a raiz quadrada de um número.

Você poderia desenvolver toda a lógica para isso.

Mas... por que fazer esse trabalho se o Python já oferece uma solução pronta? Veja o exemplo:

```
import math  
  
resultado = math.sqrt(81)  
  
print(resultado)
```

Resultado: 9.0

Percebeu? Com apenas uma linha conseguimos calcular a raiz quadrada.

A biblioteca math também oferece diversas outras funções úteis. Por exemplo:

- Potenciação;
- Seno;
- Cosseno;
- Tangente;
- Logaritmos;
- Constantes matemáticas, como o valor de π .

Não sei você, mas eu tive que estudar tudo isso na faculdade.

A biblioteca random

Você está desenvolvendo um jogo e precisa sortear um número entre 1 e 10. Como fazer isso?

Mais uma vez, não precisamos criar toda a lógica. A biblioteca random resolve esse problema.

```
import random

numero = random.randint(1, 10)

print(numero)
```

Resultado: 7

O número exibido poderá ser diferente cada vez que o programa for executado. Esse tipo de recurso aparece em diversas aplicações, como:

- Jogos;
- Sorteios;
- Simulações;
- Testes;
- Sistemas que precisam gerar valores aleatórios.

A biblioteca datetime

Você deseja exibir a data atual. Parece simples, mas calcular dia, mês, ano, horário e fuso horário não é uma tarefa tão fácil. Felizmente, já existe uma biblioteca pronta para isso.

```
from datetime import datetime

agora = datetime.now()

print(agora)
```

Resultado: 2026-08-10 10:30:45

O horário exibido dependerá do momento em que o programa for executado. No meu caso, agora.

Esse recurso é muito utilizado em sistemas de cadastro, emissão de relatórios, controle de acesso e registro de atividades.

A biblioteca os

Agora imagine que seu programa precise criar uma pasta ou descobrir o diretório atual do computador. Existe biblioteca para isso também. Ela se chama os.

Sim, OS!

Veja um exemplo logo abaixo:

```
import os  
print(os.getcwd())
```

Resultado: C:\Projetos\Python

Esse comando informa o diretório onde o programa está sendo executado.

A biblioteca os possui muitos outros recursos relacionados ao sistema operacional.

Como saber qual biblioteca utilizar?

Essa é uma dúvida muito comum. E a boa notícia é: você não precisa decorar todas elas.

Na verdade, nenhum desenvolvedor faz isso. Você não precisa decorar nada!

Com o tempo, você aprenderá quais bibliotecas são mais úteis para os tipos de projetos que desenvolve.

O importante é saber que elas existem.

Quando surgir um problema, vale a pena pesquisar se o Python já possui uma solução pronta. Na maioria das vezes...

A resposta será sim.

Um exemplo do dia a dia

Pense em um marceneiro. Ele possui uma caixa cheia de ferramentas.

- Martelo.
- Furadeira.
- Trena.
- Serrote.

Ele não utiliza todas ao mesmo tempo. Cada ferramenta é escolhida de acordo com o trabalho que precisa ser realizado.

As bibliotecas funcionam exatamente da mesma forma. Cada uma foi criada para resolver um tipo específico de problema. E se prepara, pois você irá receber vários problemas!

💬 Uma dica da Carol

Quero deixar uma mensagem antes de encerrarmos este assunto.

É muito comum quem está começando pensar algo como:

"Nossa... existe muita biblioteca. Nunca vou conseguir decorar tudo isso."

Se esse pensamento passou pela sua cabeça, relaxa. Eu também não lembro de várias.

Nem os programadores mais experientes decoram todas as bibliotecas do Python. Não tem a menor necessidade.

O que faz diferença não é saber tudo de memória, mas saber que esses recursos existem e entender quando utilizá-los.

Pode confiar.

Conforme você criar novos projetos, essas bibliotecas vão aparecendo naturalmente e, quando perceber, muitas delas já farão parte da sua rotina. Sente a vibe boa!

TRATAMENTO DE ERROS (TRY E EXCEPT)

Se você já digitou um código em Python e apareceu uma mensagem enorme em vermelho na tela... parabéns!

Isso significa que você está programando.

Sério. Todo programador já passou por isso.

Na verdade, continua passando. Não se sinta especial achando que só acontece com você, porque não é verdade.

Os erros fazem parte do desenvolvimento. A diferença é que, com o tempo, aprendemos a tratá-los da melhor forma possível.

Mas deixa eu te fazer uma pergunta.

Imagine que você desenvolveu um sistema para calcular a idade de um usuário. O programa pede que ele informe a idade. Tudo certo.

Agora imagine que, em vez de digitar um número, o usuário escreva: vinte

O que acontecerá?

Vamos descobrir.

Quando um erro acontece

Como eu disse, mas vale reforçar: erros acontecem. Não desanime ou pare na

primeira linha vermelha que surgir! Veja este exemplo:

```
idade = int(input("Digite sua idade: "))  
print("Sua idade é:", idade)
```

Se o usuário informar: 25

Tudo funcionará normalmente. Mas agora imagine que ele digite:

vinte e cinco

Nesse caso, o Python exibirá uma mensagem de erro e encerrará o programa. Sem mais ou menos.

Como sempre, você deve pensar: “Como assim, Carol?”

O problema não foi o código. O problema foi o dado informado pelo usuário.

E pode acreditar... Isso acontece o tempo todo em sistemas reais.

As pessoas digitam letras onde deveriam informar números. Esquecem informações. Digitam valores inválidos. Fecham janelas inesperadamente.

E o programa precisa estar preparado para lidar com essas situações.

É aí que entra o try. O try significa algo como: **"Tente executar este código."**

Se tudo der certo, o programa continuará normalmente. Veja um exemplo.

try:

```
idade = int(input("Digite sua idade: "))  
  
print("Sua idade é:", idade)
```

Até aqui não parece existir nenhuma diferença. E realmente não existe. A diferença aparece quando ocorre um erro.

O papel do except

Agora vamos completar nosso código.

try:

```
idade = int(input("Digite sua idade: "))  
  
print("Sua idade é:", idade)
```

except:

```
print("Valor inválido. Digite apenas números.")
```

Agora imagine novamente que o usuário informe: vinte

Resultado: “Valor inválido. Digite apenas números.”

Percebeu o que aconteceu? Em vez de exibir uma mensagem enorme de erro e encerrar o programa, mostramos uma mensagem muito mais amigável para o usuário.

E o melhor. O programa não "quebrou".

Uma comparação que ajuda bastante

Imagine que você está dirigindo e no meio do caminho aparece um quebra-molas.

Você tem duas opções: passar em alta velocidade e correr o risco de quebrar algo no carro ou reduzir a velocidade e continuar a viagem normalmente.

O tratamento de erros faz exatamente isso.

Quando algo inesperado acontece, ele evita que o programa "bata de frente" com o problema.

Em vez disso, ele encontra uma forma segura de continuar.

Outro exemplo

Imagine um programa que realiza uma divisão.

try:

```
numero = int(input("Digite um número: "))
```

```
resultado = 100 / numero
```

```
print(resultado)
```

except:

```
print("Não foi possível realizar esse cálculo.")
```

Se o usuário informar: 0

O Python normalmente encerraria o programa, pois não é possível dividir um número por zero.

Mas, graças ao try e ao except, conseguimos tratar essa situação de maneira muito mais elegante.

Posso usar try em qualquer lugar?

Tecnicamente, sim. Mas calma, porque isso não significa que devemos colocar tudo dentro de um try. O ideal é utilizá-lo apenas em trechos do programa que realmente possam gerar erros.

- Conversão de dados;
- Leitura de arquivos;
- Acesso a bancos de dados;
- Consumo de APIs;
- Operações matemáticas específicas.

Em outras palavras, utilizamos o tratamento de erros quando sabemos que existe a possibilidade de algo dar errado.

Uma outra conversa antes de terminar...

Quero que você guarde uma ideia. Quando aparecer uma mensagem de erro no Python, não pense:

"Meu programa está horrível."

Pense assim:

"O Python está tentando me mostrar exatamente onde preciso corrigir alguma coisa."

Pode parecer bobagem, mas essa mudança de pensamento faz toda a diferença. Você não é "burro", inútil ou incapaz. Até os desenvolvedores mais experientes encontram erros todos os dias.

A diferença é que eles aprenderam a enxergar essas mensagens como uma ajuda, e não como um obstáculo.

Então, quando um erro aparecer na sua tela... respira, leia a mensagem com calma e continue.

Pode confiar.

Você está aprendendo.

PROJETO PRÁTICO

Sistema de Cadastro de Produtos

Objetivo: desenvolver um sistema organizado utilizando funções, módulos da biblioteca padrão e tratamento de erros.

Chegou a hora de colocar a mão na massa.

Imagine que você foi contratado para desenvolver um pequeno sistema de cadastro para uma loja. O programa deverá permitir que o usuário cadastre diversos produtos e, ao final, exiba um relatório com todas as informações registradas.

Durante esse projeto, utilizaremos praticamente todos os conceitos aprendidos.

Pode parecer bastante coisa (e é) e que você não vai conseguir, mas respira!

Você vai perceber que um conceito complementa o outro.

Desafio

Desenvolva um programa que seja capaz de:

- Cadastrar produtos;
- Solicitar nome, preço e quantidade;
- Utilizar funções para organizar o código;
- Retornar informações utilizando return;
- Tratar possíveis erros de digitação utilizando try e except;
- Armazenar todos os produtos em uma lista;
- Exibir um relatório final.

TENTE ANTES DE VER A SOLUÇÃO!

Solução

Lista que armazenará os produtos

produtos = []

def cadastrar_produto():

print("\n===== NOVO PRODUTO =====")

nome = input("Nome do produto: ")

while True:

try:

preco = float(input("Preço: R\$ "))

quantidade = int(input("Quantidade: "))

break

except:

print("Dados inválidos. Tente novamente.")

return {

"nome": nome,

```
"preco": preco,  
  
"quantidade": quantidade  
  
}
```

```
def exibir_relatorio(lista_produtos):
```

```
    print("\n==== RELATÓRIO =====")
```

```
    for produto in lista_produtos:
```

```
        print("-----")
```

```
        print("Produto:", produto["nome"])
```

```
        print("Preço: R$", produto["preco"])
```

```
        print("Quantidade:", produto["quantidade"])
```

```
        print("-----")
```

```
    print("Total de produtos cadastrados:", len(lista_produtos))
```

```
continuar = "S"
```

```
while continuar == "S":
```

```
    produto = cadastrar_produto()
```

```
    produtos.append(produto)
```

```
    continuar = input("\nDeseja cadastrar outro produto? (S/N): ").upper()
```

```
exibir_relatorio(produtos)
```

O que aprendemos neste projeto?

Olha quanta coisa utilizamos em um único programa.

- Funções para dividir responsabilidades;
- Utilização de parâmetros em funções;
- return para devolver dados;
- Variáveis locais;
- Biblioteca padrão (len());
- Tratamento de erros utilizando try e except;
- Listas;
- Dicionários;
- Laços de repetição;
- Estruturas condicionais.

E nenhum conceito apareceu sozinho.

Desafio Extra

Agora é com você. Tente deixar esse sistema ainda mais completo. Algumas ideias para isso:

- Criar uma função para pesquisar um produto;

- Permitir alterar o preço de um produto;
- Remover produtos cadastrados;
- Calcular automaticamente o valor total do estoque;
- Exibir o produto mais caro da lista;
- Mostrar o valor médio dos produtos cadastrados.

E saiba que não existe apenas uma solução. Esse é justamente o lado mais divertido da programação. Sempre existe uma forma de melhorar mais ainda o sistema.

AGRADECIMENTO

Se você chegou até aqui, quero te dizer: obrigada.

De verdade.

Finalizar um material assim não é só sobre aprender conteúdo. É sobre dedicação, tempo e vontade de evoluir. E só de você estar aqui, já mostra muito sobre você.

Essa apostila foi criada com um propósito muito claro: mostrar que conhecimento não precisa ser complicado. Que assuntos que parecem difíceis podem, sim, ser entendidos quando explicados da forma certa.

E mais do que isso: que conhecimento deve ser para todos.

Por isso, este material também foi pensado para ser acessível. Ele conta com versão em Braille, porque inclusão não é um diferencial — é o mínimo.

Todo mundo merece aprender. Todo mundo merece ter acesso.

Se, de alguma forma, esse conteúdo te ajudou a entender melhor, a se organizar ou até a enxergar as coisas de um jeito diferente, então ele já cumpriu o seu papel.

Obrigada por fazer parte disso.

E lembre-se: você não precisa complicar para fazer dar certo.

CONTATO

Se você quiser continuar aprendendo, trocar ideias ou acompanhar mais conteúdos sobre projetos, tecnologia e organização no dia a dia, você pode me encontrar por aqui:

Instagram: @CAROL_G_INTECH

LinkedIn: Caroline Gonçalves

Site: <https://cpgconsulting.com.br/>

E-mail: cpgtechconsulting@gmail.com

Fique à vontade para se conectar ou mandar uma mensagem. Vou adorar continuar essa troca com você.